

Problem Description:

Tina owns a match making company, which even to her surprise is an extreme hit. She says that her success rate cannot be matched (Yes, letterplay!) in the entire match-making industry. She follows an extremely simple algorithm to determine if two people are matches for each other. Her algorithm is not at all complex, and makes no sense - not even to her. But she uses it anyway.

Let's say that on a given day she decides to select n people - that is, n boys and n girls. She gets the list of n boys and n girls in a random order initially. Then, she arranges the list of girls in ascending order on the basis of their height and boys in descending order of their heights. A girl A_i can be matched to a boy on the same index only, that is, B_i and no one else. Likewise, a girl standing on A_k can be only matched to a boy on the same index B_k and no one else.

Now to determine if the pair would make an ideal pair, she checks if the modulo of their heights is 0, i.e., $A_i \% B_i == 0$ or $B_i \% A_i == 0$. Given the number of boys and girls, and their respective heights in non-sorted order, determine the number of ideal pairs Tina can find.

Constraints:

1 <= Test Cases <= 10^2

1 <= N <= 10^4

1 <= A_i , B_i <= 10^5

Input Format:

The first line contains number of test cases. Then, the next line contains an integer, n , saying the number of boys and girls. The next line contains the height of girls, followed by the height of boys.

Output Format:

Print the number of ideal pairs in a separate lines

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

2
4
1 6 9 12
4 12 3 9
4
2 2 2 2
2 2 2 2

EXPECTED OUTPUT

2
4

Test Case 2

INPUT (STDIN)

2
4
3 6 9 12
4 12 16 9
4
1 1 1 1
2 2 2 2

EXPECTED OUTPUT

3
4

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(int i = 0;i<n;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

7

Test Case 2

TOKEN COUNT

210

Test Case 3

NLOC

23

```
Sorting1.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int cmp_asc(const void *a, const void *b) {
5     return (*(int*)a - *(int*)b);
6 }
7
8 int cmp_desc(const void *a, const void *b) {
9     return (*(int*)b - *(int*)a);
10}
11
12int main() {
13    int t;
14    if (scanf("%d", &t) != 1) return 0;
15    while (t--) {
16        int n;
17        scanf("%d", &n);
18        int a[10005], b[10005];
19        for (int i = 0; i < n; i++) {
20            scanf("%d", &a[i]);
21        }
22        for (int i = 0; i < n; i++) {
23            scanf("%d", &b[i]);
24        }
25        qsort(a, n, sizeof(int), cmp_asc);
```

```
26     qsort(b, n, sizeof(int), cmp_desc);
27     int count = 0;
28     for (int i = 0; i < n; i++) {
29         if (a[i] % b[i] == 0 || b[i] % a[i] == 0) {
30             count++;
31         }
```

```
32     }  
33     printf("%d\n", count);  
34 }  
35 return 0;  
36}  
37
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

2

4

1 6 9 12

4 12 3 9

4

2 2 2 2

2 2 2 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Sorting1.exe

2

4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

2

4

3 6 9 12

4 12 16 9

4

1 1 1 1

2 2 2 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Sorting1.exe

3

4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>